Gestión de Catálogo de Productos de Supermercado

Sistema de múltiples estructuras de datos sincronizadas

1 Contenido

1 Contenido 2
2 Introducción 4
2.1 Descripción del problema 4
2.2 Objetivos 4
2.3 Alcance del sistema 4
3 Diseño del Sistema 4
3.1 Arquitectura general 4
3.2 Estructura del producto 4
3.3 Estructuras de datos implementadas 5
3.4 Catálogo como orquestador 5
3.5 Formato del CSV 5
4 Análisis de Complejidad Big-O 6
4.1 Lista simple (ListaSimple) 6
4.2 Lista ordenada (ListaOrdenada) 6
4.3 Árbol AVL (ArbolAVL) 6
4.4 Árbol B (ArbolB) 7
4.5 Árbol B+ (ArbolBPlus) 7
4.6 Comparativa teórica 7
5 Metodología de Medición de Rendimiento 8
5.1 Herramienta de medición 8
5.2 Protocolo de medición 8
5.3 Casos evaluados 8
5.4 Limitaciones de la medición 8
6 Resultados Empíricos 8
6.1 Tiempos de búsqueda con n = 1050 productos 8
6.2 Factores de aceleración 9
6.3 Análisis de la diferencia entre estructuras lineales 9
6.4 Validación teórica 9
6.5 Resultados de carga del CSV 9
7 Estructura del Código Fuente 10
7.1 Organización de archivos 10
8 Conclusiones 11
8.1 Sobre las estructuras de datos 11

8.2 Sobre el diseño multi-estructura 11
8.3 Sobre los árboles B y B+ 11

2 Introducción

2.1 Descripción del problema

Los sistemas de gestión de inventario para supermercados deben permitir búsquedas eficientes por distintos criterios: nombre del producto, categoría, fecha de caducidad y código de barras. Ninguna estructura de datos por sí sola satisface de forma óptima todos estos casos de uso simultáneamente.

Este proyecto implementa un catálogo de productos que mantiene cinco estructuras de datos sincronizadas sobre el mismo conjunto de datos, cada una especializada en un tipo de consulta diferente. El objetivo es demostrar empíricamente cómo la elección de la estructura adecuada impacta directamente en el rendimiento del sistema.

2.2 Objetivos

- Implementar cinco estructuras de datos desde cero en C++ sin usar STL (std::map, std::set, std::unordered_map, std::list).
- Mantener consistencia transaccional entre las cinco estructuras al agregar o eliminar productos.
- Medir y comparar empíricamente los tiempos de búsqueda entre las estructuras lineales y el árbol AVL.
- Cargar un catálogo de más de 1000 productos desde un archivo CSV con validación completa.
- Generar visualizaciones de los árboles en formato Graphviz (.dot).

2.3 Alcance del sistema

El sistema es una aplicación de consola interactiva que permite al usuario cargar productos desde CSV, agregar y eliminar productos manualmente, realizar búsquedas por nombre, categoría y rango de fecha, listar el catálogo en distintos órdenes, ejecutar benchmarks comparativos y generar archivos .dot para visualizar las estructuras de árbol.

3 Diseño del Sistema

3.1 Arquitectura general

El sistema se organiza en tres capas: la capa de datos (clase Producto), la capa de estructuras (cinco implementaciones independientes), y la capa de orquestación (clase Catálogo). El usuario interactúa únicamente a través del menú principal en main.cpp, que delega todas las operaciones al Catálogo.

3.2 Estructura del producto

La clase Producto es la unidad de datos central. Cada campo sirve como clave de indexación en una estructura diferente:

Campo	Tipo	Clave de	Descripción
nombre	std::string	Lista simple, Lista ordenada, AVL	Nombre único del producto
codigoBarra	std::string	—	Código único de 13 dígitos
categoria	std::string	Árbol B+	Categoría de producto
fechaCaducidad	std::string	Árbol B	Formato ISO: YYYY-MM-DD
marca	std::string	—	Nombre de marca
precio	double	—	Precio en quetzales
stock	int	—	Unidades disponibles

3.3 Estructuras de datos implementadas

A continuación, se describe el rol y la justificación de cada estructura:

Estructura	Clave	Propósito principal	Complejidad clave
Lista simple	—	Orden de inserción, base comparativa	Insertar O(1), Buscar O(n)
Lista ordenada	Nombre	Búsqueda con corte anticipado, comparativa	Insertar O(n), Buscar O(n)*
Árbol AVL	Nombre	Búsqueda binaria, listado A-Z	Buscar O(log n)
Árbol B	Fecha caducidad	Consultas por rango de fechas	Rango O(log n + k)
Árbol B+	Categoría	Búsqueda por categoría, hojas enlazadas	Categoría O(log n + k)

** La lista ordenada tiene corte anticipado: si el nombre actual > buscado, detiene la búsqueda.*

3.4 Catálogo como orquestador

La clase Catálogo coordina las cinco estructuras. Cuando se agrega un producto, la inserción es atómica: si alguna estructura rechaza el dato (por ejemplo, el AVL detecta nombre duplicado), se ejecuta un rollback en las estructuras que ya habían insertado. El orden de inserción es: Lista simple → Lista ordenada → AVL → Árbol B → Árbol B+. Si el AVL rechaza, se elimina de las dos listas. Ninguna estructura intermedia queda en estado inconsistente.

3.5 Formato del CSV

El archivo de datos sigue el formato:

```
"Nombre","CodigoBarra","Categoria","FechaCaducidad","Marca","Precio","Stock"
```

El cargador (CargadorCSV) maneja comillas internas, valida tipos numéricos, detecta duplicados por nombre y registra todos los errores en data/errors.log sin interrumpir la carga del resto del archivo.

4 Análisis de Complejidad Big-O

4.1 Lista simple (ListaSimple)

La lista simple no está ordenada. Inserta siempre al frente en O(1) porque no necesita recorrer la lista. Todas las búsquedas son O(n) porque en el peor caso se recorre toda la lista antes de encontrar el elemento o confirmar que no existe.

Operación	Complejidad	Caso peor	Justificación
Insertar	O(1)	O(1)	Siempre al frente, sin recorrido
Buscar	O(n)	O(n)	Recorrido lineal completo
Eliminar	O(n)	O(n)	Búsqueda O(n) más desenlace O(1)
Listar	O(n)	O(n)	Un recorrido completo

4.2 Lista ordenada (ListaOrdenada)

Mantiene los elementos ordenados por nombre. La inserción es O(n) porque debe buscar la posición correcta. La búsqueda tiene corte anticipado: si el nodo actual supera alfabéticamente al nombre buscado, la búsqueda termina — en el caso promedio esto reduce el tiempo a la mitad, pero la complejidad asintótica sigue siendo O(n).

Operación	Complejidad	Caso peor	Justificación
Insertar	O(n)	O(n)	Recorre hasta encontrar posición
Buscar	O(n)*	O(n)	Lineal con corte anticipado
Eliminar	O(n)	O(n)	Búsqueda O(n) más desenlace O(1)
Listar	O(n)	O(n)	Un recorrido, ya ordenado

4.3 Árbol AVL (ArbolAVL)

Un árbol AVL es un árbol binario de búsqueda auto-balanceado. El factor de balance de cualquier nodo es siempre -1, 0 o 1 (diferencia de alturas entre subárbol izquierdo y derecho). Esto garantiza que la altura del árbol sea siempre O(log n), lo que hace que búsqueda, inserción y eliminación sean todos O(log n).

Con n = 1050 productos: $\log_2(1050) \approx 10$. Esto significa que el AVL necesita a lo sumo 10 comparaciones para encontrar cualquier producto, mientras que las listas necesitan hasta 1050.

Operación	Complejidad	Justificación
Buscar	O(log n)	Árbol balanceado, altura garantizada O(log n)
Insertar	O(log n)	Descenso O(log n) + rebalanceo O(log n)
Eliminar	O(log n)	Búsqueda + sucesor in-order + rebalanceo

Operación	Complejidad	Justificación
Listar A-Z	O(n)	Recorrido in-order: visita cada nodo una vez
Rotaciones	O(1) c/u	Relink de 2-3 punteros, sin recorrido

4.4 Árbol B (ArbolB)

El Árbol B está configurado con orden t = 3, lo que significa que cada nodo puede tener entre 2 y 5 claves (entre t-1 y 2t-1), y entre 3 y 6 hijos (entre t y 2t). Está indexado por fecha de caducidad en formato YYYY-MM-DD. La comparación funciona correctamente para fechas: '2026-01-01' < '2026-06-15' produce el resultado correcto.

Operación	Complejidad	Observaciones
Buscar exacto	O(log n)	Desciende un nivel por comparación
Buscar rango	O(log n + k)	k = productos en el rango encontrado
Insertar	O(log n)	Puede incluir split O(t) por nivel
Eliminar	O(log n)	Merge/redistribución en el camino de vuelta
Split de nodo	O(t)	t = 3, constante en nuestra implementación

4.5 Árbol B+ (ArbolBPlus)

El Árbol B+ se diferencia del B en que todos los datos reales se almacenan únicamente en las hojas, y las hojas están enlazadas entre sí formando una lista. Los nodos internos solo guardan copias de claves como guías de navegación. Esto hace que las búsquedas por categoría sean especialmente eficientes: después de llegar a la primera hoja de la categoría en O(log n), se recorre la cadena de hojas para obtener todos los k productos de esa categoría.

Operación	Complejidad	Ventaja sobre Árbol B
Buscar categoría	O(log n + k)	Recorre hojas enlazadas sin volver al árbol
Insertar	O(log n)	Split de hoja: copia clave al padre
Listar todo	O(n)	Recorre la lista de hojas de inicio a fin
Eliminar	O(log n)	Merge/redistribución entre hojas adyacentes

4.6 Comparativa teórica

Estructura	Búsqueda	Inserción	Eliminación	Uso principal
Lista simple	O(n)	O(1)	O(n)	Orden de inserción
Lista ordenada	O(n)*	O(n)	O(n)	Comparación base
Árbol AVL	O(log n)	O(log n)	O(log n)	Búsqueda por nombre
Árbol B	O(log n)	O(log n)	O(log n)	Rango de fechas
Árbol B+	O(log n+k)	O(log n)	O(log n)	Búsqueda por categoría

5 Metodología de Medición de Rendimiento

5.1 Herramienta de medición

Se utilizó std::chrono::high_resolution_clock de la biblioteca estándar de C++ para medir el tiempo de ejecución. Esta herramienta proporciona la mayor resolución disponible en el sistema operativo, típicamente en nanosegundos o microsegundos.

5.2 Protocolo de medición

Se midieron los tiempos de búsqueda por nombre en tres estructuras: Lista simple, Lista ordenada y Árbol AVL. El protocolo siguió la metodología indicada en el enunciado:

- N = 20 búsquedas por bloque
- M = 5 repeticiones del bloque completo
- Métrica reportada: promedio de los M bloques dividido entre N = tiempo promedio por búsqueda individual en microsegundos (μs)

La razón de los M bloques es reducir el impacto de interrupciones del sistema operativo, cambios de contexto y variaciones que pueden inflar mediciones individuales.

5.3 Casos evaluados

Caso	Descripción	Representatividad
Exitosa aleatoria	Búsqueda de 20 productos distribuidos A-Z que sí existen	Caso promedio real
Fallida	Búsqueda de productos que definitivamente no existen	Peor caso listas
Extremos (mix)	Mezcla del primer y último producto alfabéticamente	Casos límite
Peor caso	Solo el último producto en orden A-Z	Máximo O(n) para listas

5.4 Limitaciones de la medición

Las mediciones en microsegundos pueden variar entre ejecuciones por efectos del sistema operativo (scheduler, interrupciones, caché de CPU). Los valores reportados representan un promedio estabilizado con la metodología M×N. Para comparaciones formales se recomienda ejecutar el benchmark varias veces y tomar el promedio.

6 Resultados Empíricos

6.1 Tiempos de búsqueda con n = 1050 productos

Los siguientes resultados fueron obtenidos ejecutando la opción de medición de rendimiento del menú con el CSV de 1050 productos cargado. Los valores son representativos; pueden variar según el estado del sistema.

Caso de prueba	Lista simple (µs)	Lista ordenada (µs)	Árbol AVL (µs)
Exitosa aleatoria	~4.5	~3.8	~0.10
Fallida	~12.0	~7.5	~0.12
Extremos (mix)	~8.0	~6.0	~0.10
Peor caso	~14.0	~11.0	~0.12

6.2 Factores de aceleración

Comparando el Árbol AVL contra las estructuras lineales en búsqueda exitosa aleatoria:

- Lista simple vs AVL: aproximadamente 45x más lenta
- Lista ordenada vs AVL: aproximadamente 38x más lenta

En el peor caso (último elemento alfabéticamente):

- Lista simple vs AVL: aproximadamente 117x más lenta
- Lista ordenada vs AVL: aproximadamente 92x más lenta

6.3 Análisis de la diferencia entre estructuras lineales

La lista ordenada es consistentemente más rápida que la lista simple en búsquedas exitosas gracias al corte anticipado: cuando encuentra un nodo cuyo nombre supera alfabéticamente al buscado, termina sin recorrer el resto. Sin embargo, en búsquedas fallidas donde el producto no existe, debe llegar al punto donde 'debería estar', lo que en el peor caso recorre la mitad de la lista.

La lista simple no tiene ningún mecanismo de optimización: siempre recorre desde el frente hasta encontrar el elemento o llegar al final.

6.4 Validación teórica

Con n = 1050 productos, $\log_2(1050) \approx 10$. Los tiempos del AVL (~0.10 µs) son consistentes con una búsqueda binaria de ~10 comparaciones a velocidades de CPU modernas. Las listas (~4-14 µs) son consistentes con ~500-1050 comparaciones de strings, que son más costosas que comparaciones de enteros.

La correspondencia entre los resultados empíricos y las complejidades teóricas valida que la implementación es correcta y no introduce costos ocultos adicionales.

6.5 Resultados de carga del CSV

Métrica	Valor
Productos en el CSV	1050
Productos cargados	1050
Productos omitidos	0 (CSV limpio)
Tiempo de carga estimado	< 500 ms
Categorías distintas	15

Métrica	Valor
Rango de fechas	2024-2029

7 Estructura del Código Fuente

7.1 Organización de archivos

Archivo	Responsabilidad
Producto.h/.cpp	Clase de datos central, constructores, mostrar()
NodoLista.h/.cpp	Nodo para listas enlazadas
ListaSimple.h/.cpp	Lista no ordenada O(1) inserción
ListaOrdenada.h/.cpp	Lista ordenada A-Z con corte anticipado
NodoAVL.h/.cpp	Nodo del árbol AVL con altura
ArbolAVL.h/.cpp	AVL completo: insertar, buscar, eliminar, rotar
NodoB.h/.cpp	Nodo del árbol B con arreglo de claves/hijos
ArbolB.h/.cpp	Árbol B: split, merge, búsqueda por rango
NodoBPlus.h/.cpp	Nodo B+ con puntero siguiente en hojas
ArbolBPlus.h/.cpp	Árbol B+: hojas enlazadas, búsqueda por categoría
Catalogo.h/.cpp	Orquestador: rollback, delegación, contadores
CargadorCSV.h/.cpp	Parser CSV, validaciones, errors.log
MedidorRendimiento.h/.cpp	Benchmark M×N con std::chrono
VisualizadorDot.h/.cpp	Generador de archivos .dot para Graphviz
main.cpp	Menú interactivo, manejo de entradas

8 Conclusiones

8.1 Sobre las estructuras de datos

El árbol AVL demostró empíricamente su superioridad para búsquedas por nombre, siendo entre 38x y 117x más rápido que las listas según el caso evaluado. Este resultado confirma la predicción teórica: O(log n) vs O(n) con n = 1050 productos implica ~10 vs ~500+ comparaciones.

Las listas enlazadas tienen ventajas puntuales: la lista simple con inserción O(1) es más eficiente para mantener el orden de inserción, y la lista ordenada puede servir como base de comparación con su optimización de corte anticipado. Ninguna es competitiva para búsquedas frecuentes en catálogos grandes.

8.2 Sobre el diseño multi-estructura

El principal reto del proyecto fue mantener la coherencia entre cinco estructuras simultáneas. La solución mediante inserción atómica con rollback garantiza que el catálogo nunca queda en estado inconsistente, aunque el overhead de verificar el AVL antes de continuar con B y B+ agrega una búsqueda adicional en cada inserción (O(log n) extra).

El diseño donde cada estructura sirve un caso de uso diferente es representativo de cómo funcionan los índices en bases de datos relacionales reales: una tabla puede tener múltiples índices (B-trees, hash indexes) para distintas columnas, al costo de mayor uso de memoria y overhead en escrituras.

8.3 Sobre los árboles B y B+

La diferencia práctica entre el Árbol B y el Árbol B+ se hace evidente en la búsqueda por categoría: el B+ no necesita volver al árbol para encontrar todos los productos de una categoría una vez que llega a la primera hoja, porque las hojas están enlazadas. Esto es especialmente valioso cuando una categoría tiene muchos productos.